Aprendizaje Automático - Libro 1 - Clase 1 a 7

3. Clase 3

Clase 3 · Preparación de datos para el aprendizaje automático

1. Presentación

Bienvenidos a la clase 3. Antes de pedirle a un modelo que aprenda, necesitamos decidir cómo representar los datos que recibirá. Una tabla puede contener importes, edades, ciudades, niveles educativos, valores faltantes y errores de escritura. Cada elemento requiere una decisión: no corresponde transformar todas las columnas de la misma manera.

En esta clase aprenderemos a inspeccionar, limpiar, completar, escalar y codificar datos. Trabajaremos con ejemplos pequeños y luego integraremos las transformaciones en un proceso reproducible. Todavía no entrenaremos modelos: el resultado será una matriz de entrada preparada para utilizarlos en las próximas clases.

Objetivos de aprendizaje

- Diferenciar variables de entrada y variable objetivo.
- Reconocer variables numéricas, categóricas nominales, ordinales y binarias.
- Elegir entre reescalado, estandarización, escalado robusto, normalización y binarización.
- Convertir categorías en representaciones numéricas sin inventar un orden.
- Explicar ventajas y limitaciones de cada transformación.
- Ajustar el preprocesamiento con entrenamiento y reutilizarlo en prueba y datos nuevos.

Pregunta inicial: si una tabla contiene edad, ingreso y ciudad, ¿qué problemas aparecerían si intentáramos operar matemáticamente con ella tal como está?

2. Qué significa preparar los datos

El preprocesamiento transforma los datos originales en una representación compatible con el problema y con el algoritmo que se utilizará. No garantiza buenas predicciones ni reemplaza la selección de información relevante.

Una secuencia de trabajo habitual es:

1. Definir qué se quiere predecir y qué información estará disponible al hacerlo.
2. Identificar tipos de variables y aplicar reglas de calidad conocidas.
3. Separar las entradas x de la salida y y reservar datos de prueba.
4. Aprender del entrenamiento los valores de imputación, escalas y categorías.
5. Aplicar esas mismas transformaciones a entrenamiento, prueba y datos nuevos.
6. Revisar la matriz resultante antes de entrenar un modelo.

No hay una receta universal. Un escalador, por ejemplo, puede ser importante para un procedimiento basado en distancias y prescindible para un árbol. Tampoco se aplican todos los escaladores uno detrás de otro: normalmente son alternativas.

3. Variables de entrada y variable objetivo

x contiene las características disponibles para realizar una predicción. y es aquello que queremos predecir en aprendizaje supervisado.

Problema	Entradas x	Objetivo y	Tipo de tarea
Predecir gasto del próximo mes	Edad, ingreso, ciudad, nivel de relación con el servicio	Gasto futuro en pesos	Regresión
Predecir si una persona renovará un servicio	Las mismas características	Renueva / no renueva	Clasificación binaria
Predecir el canal que elegirá	Edad, compras previas, ciudad	Web / local / teléfono	Clasificación multiclase

La tarea se define por el significado de `y`, no por el tipo de columnas de `X`. Tener una ciudad entre las entradas no convierte el problema en clasificación. Codificar una clase como 0 o 1 tampoco convierte la clasificación en regresión.

En tareas no supervisadas puede no existir `y`; aun así, es necesario preparar `X`.

3.1 Tipos de variables

Tipo	Ejemplo	Qué significa	Preparación posible
Numérica continua	Ingreso, altura	La magnitud y las diferencias tienen significado	Imputación y escalado según el modelo
Numérica discreta	Número de compras	Es un conteo	Conservar el conteo; escalar o binarizar si el objetivo lo justifica
Categórica nominal	Ciudad, canal	Categorías sin jerarquía	One-hot o tratamiento categórico nativo
Categórica ordinal	Bajo, medio, alto	Existe un orden, sin distancias necesariamente iguales	Codificación ordinal explícita; evaluar su interpretación
Categórica binaria	Sí / no	Dos categorías	Mapeo explícito 0/1
Identificador	Número de cliente	Identifica, no mide	Habitualmente excluir de las entradas

El tipo de almacenamiento no alcanza para interpretar una variable. Un código postal puede estar escrito con números y seguir siendo categórico. Un importe leído como texto puede ser una variable numérica mal importada.

4. Inspección, limpieza e imputación

4.1 Inspeccionar antes de transformar

Con pandas usamos `head()`, `info()`, `describe()`, `isna().sum()`, `value_counts()` y `duplicated()` para reconocer tipos, rangos, faltantes, categorías y repeticiones. Conviene mirar también gráficos y consultar el significado de cada columna.

Buscamos edades imposibles, unidades mezcladas, importes como texto, etiquetas inconsistentes, categorías poco frecuentes y datos que todavía no existirían al momento de predecir.

Ventaja: permite detectar problemas que un escalador no soluciona. **Limitación:** una estadística aislada no explica la causa del problema; un valor extremo puede ser legítimo.

4.2 Corregir formatos y errores conocidos

`pd.to_numeric(..., errors="coerce")` convierte texto numérico y transforma entradas inválidas en faltantes. `str.strip()` elimina espacios externos y `str.lower()` unifica mayúsculas. `replace()` permite corregir equivalencias explícitas. `drop_duplicates()` elimina repeticiones cuando representan duplicación accidental de la misma observación.

Ventajas: recuperan tipos correctos y evitan categorías artificiales como `Web` y `web`. **Desventajas:** pueden borrar diferencias legítimas o convertir errores en faltantes sin resolver su origen. Antes de eliminar duplicados, confirmar la unidad de observación: dos compras iguales pueden ser dos operaciones válidas.

Las reglas conocidas, como rechazar una edad negativa, pueden definirse antes de separar datos. Las decisiones aprendidas de la distribución —por ejemplo, límites basados en percentiles— se obtienen exclusivamente del entrenamiento.

4.3 Completar faltantes: `SimpleImputer()`

Imputar significa sustituir un dato faltante mediante una regla. Primero se investiga por qué falta. No se debe reemplazar automáticamente todo por cero: cero puede significar ausencia real y no desconocimiento.

Estrategia	Aplicación	Ventaja	Desventaja
`mean`	Numéricas	Simple; usa todos los valores observados	Sensible a extremos; modifica la variabilidad
`median`	Numéricas	Menos afectada por valores extremos	Puede acumular muchos casos en un mismo valor
`most_frequent`	Categóricas o discretas	Conserva un valor existente	Refuerza la categoría mayoritaria
`constant`	Categorías o números con justificación	Hace explícito «sin dato»	Agrega una categoría o valor artificial

No se imputa `y` automáticamente para inventar respuestas de entrenamiento. Los registros sin objetivo requieren una decisión aparte. Imputar conserva filas, pero no recupera el dato verdadero ni elimina sesgos por faltantes.

```
from sklearn.impute import SimpleImputer
imputador = SimpleImputer(strategy="median")
X_train_completo = imputador.fit_transform(X_train_numerico)
X_test_completo = imputador.transform(X_test_numerico)
```

5. Por qué algunas escalas importan

Supongamos dos entradas: edad entre 18 y 80 años e ingreso entre 300.000 y 4.000.000 de pesos. En un cálculo de distancia sin ajustar escalas, una diferencia de ingresos puede dominar numéricamente una diferencia de edad. Eso sucede por las unidades utilizadas, no necesariamente por la importancia predictiva.

Los métodos basados en distancias y muchos procedimientos de optimización o regularización son sensibles a las escalas. Más adelante veremos ejemplos como vecinos cercanos, SVM, redes neuronales y modelos lineales regularizados. Los árboles suelen ser poco sensibles al escalado individual de columnas porque trabajan con umbrales. Esta orientación no sustituye la evaluación de cada caso.

Escalar no garantiza que todas las variables tengan la misma importancia, no corrige asociaciones engañosas ni resuelve valores mal registrados.

6. Reescalado al rango 0–1: `MinMaxScaler()`

Trabaja **por columna**, usando el mínimo y el máximo del entrenamiento. Para una columna no constante:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Ejemplo: `[10, 20, 30]` se convierte en `[0, 0.5, 1]`.

Cuándo considerarlo: cuando interesa un rango de referencia común o acotado en entrenamiento.

Ventajas: interpretación sencilla; conserva el orden y las proporciones entre diferencias dentro de una columna.

Desventajas: un extremo puede comprimir a casi todos los demás valores. No elimina atípicos. Los datos nuevos pueden quedar fuera de 0–1 si superan los extremos aprendidos: 40 en el ejemplo produce 1.5. `clip=True` limita el rango, pero pierde esa información.

```
from sklearn.preprocessing import MinMaxScaler
escala = MinMaxScaler()
escala.fit_transform([[10], [20], [30]])
escala.transform([[40]])  # [[1.5]]
```

Referencia: MinMaxScaler.

7. Estandarización: `StandardScaler()`

Trabaja **por columna**, resta la media del entrenamiento y divide por su desviación estándar:

$$z = \frac{x - \mu}{\sigma}$$

Para columnas no constantes, el entrenamiento transformado tiene media aproximadamente 0 y desviación estándar 1. La prueba no tiene por qué cumplir esos valores. scikit-learn utiliza la desviación con `ddof=0`.

Centrar es solo restar la media. **Estandarizar** agrega la división por la desviación. `StandardScaler(with_std=False)` permite observar el centrado por separado.

Ventajas: expresa diferencias en unidades de desviación estándar; reduce desequilibrios de escala.

Desventajas: es sensible a extremos y no acota el rango. Puede producir números negativos. En matrices dispersas, centrar destruye la dispersión; existe `with_mean=False` para evitarlo.

No vuelve gaussiana una distribución, no elimina sesgos del dataset y no iguala la relevancia de las variables. Una columna constante queda en cero con el centrado habitual.

```
from sklearn.preprocessing import StandardScaler
StandardScaler().fit_transform([[10], [20], [30]])
# Aproximadamente: -1.2247, 0, 1.2247
```

Referencia: StandardScaler.

8. Alternativa ante extremos: `RobustScaler()`

Por defecto resta la mediana y divide por el rango intercuartílico:

$$x' = \frac{x - \text{mediana}(x)}{Q_{75} - Q_{25}}$$

Ventaja: los parámetros de transformación son menos sensibles a extremos que la media y la desviación.

Desventajas: no elimina extremos, no asegura rango 0–1 ni desviación 1. Si la dispersión central es muy pequeña, los extremos transformados pueden seguir siendo enormes. La biblioteca trata escalas nulas para evitar dividir por cero.

Uso: comparar como alternativa cuando hay valores extremos válidos. Si son errores, corresponde corregirlos primero.

```
from sklearn.preprocessing import RobustScaler
RobustScaler().fit_transform([[10], [11], [12], [13], [1000]])
```

Referencia: RobustScaler.

9. Normalización de cada observación: `Normalizer()` y `normalize()`

Aquí normalizar significa dividir **cada fila** por su norma. No significa escalar cada columna al rango 0–1. Es útil cuando interesa el perfil relativo de una observación más que su tamaño absoluto; por ejemplo, conteos de distintos tipos de interacción.

Norma	Divisor	Resultado para `[3, 4]`
L1	Suma de valores absolutos: 7	`[3/7, 4/7]`
L2	Raíz de la suma de cuadrados: 5	`[0.6, 0.8]`
Máximo	Máximo valor absoluto: 4	`[0.75, 1]`

Con L1 suman 1 los valores absolutos; solo si todos son no negativos suman 1 los valores originales normalizados. Con L2 vale 1 la longitud euclídea, no la suma. Una fila de ceros permanece en cero.

Ventaja: permite comparar perfiles de distinto volumen. **Desventajas:** pierde magnitud absoluta; `[3, 4]` y `[30, 40]` terminan iguales. No conviene mezclar edad e ingreso por fila sin justificar unidades y significado.

```
from sklearn.preprocessing import normalize, Normalizer
normalize([[3, 4]], norm="l2", axis=1)
Normalizer(norm="l2").transform([[3, 4]])
```

Referencia: Normalizer.

10. Binarización: `Binarizer()`

Convierte una variable numérica en un indicador: devuelve 1 si el valor es **mayor** que el umbral y 0 si es menor o igual.

$$b(x) = \{1 \quad x > t\, 0 \quad x \leq t$$

Con umbral 0, `[0, 1, 5]` produce `[0, 1, 1]`: ninguna compra frente a alguna compra.

Ventajas: es simple e interpretable; útil cuando importa presencia o ausencia. **Desventajas:** pierde cantidades y depende del umbral elegido. Tener 1 compra y 50 se vuelve indistinguible.

```
from sklearn.preprocessing import Binarizer
Binarizer(threshold=0).transform([[0], [1], [5]])
```

Binarizar una entrada no convierte por sí mismo la tarea en clasificación. Binarización tampoco es lo mismo que one-hot: una aplica un umbral; la otra representa categorías.

Referencia: Binarizer.

11. Tratamiento de variables categóricas

Codificar es representar categorías con números. **El número resultante no siempre representa una cantidad.** Limpiar las etiquetas y resolver faltantes antes de codificar evita categorías duplicadas o ambiguas.

11.1 Ordinales: `OrdinalEncoder()`

Si existe un orden conocido, lo explicitamos: bajo → 0, medio → 1, alto → 2. No debemos dejar que el orden alfabético decida la jerarquía.

```
from sklearn.preprocessing import OrdinalEncoder
ordinal = OrdinalEncoder(
    categories=[["bajo", "medio", "alto"]],
    handle_unknown="use_encoded_value", unknown_value=-1
)
ordinal.fit_transform([["medio"], ["alto"], ["bajo"]])
```

Ventajas: mantiene una columna y conserva el orden. **Desventaja central:** en algoritmos que tratan esos códigos como magnitudes, puede introducir distancias iguales que no existen. Pasar de bajo a medio no tiene por qué equivaler a pasar de medio a alto.

El valor -1 para desconocidos es una convención técnica, no un nivel inferior real. Debe revisarse su efecto. Si el tratamiento numérico del orden no es adecuado, puede considerarse one-hot, aun perdiendo el orden explícito.

Referencia: OrdinalEncoder.

11.2 Nominales: `OneHotEncoder()`

Asignar Ushuaia = 0, Río Grande = 1 y Tolhuin = 2 puede inventar una jerarquía y distancias entre ciudades. One-hot crea un indicador por categoría.

Ciudad	ciudad_Ushuaia	ciudad_Río Grande	ciudad_Tolhuin
Ushuaia	1	0	0
Río Grande	0	1	0
Tolhuin	0	0	1

```
from sklearn.preprocessing import OneHotEncoder
nominal = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
nominal.fit_transform([["Ushuaia"], ["Tolhuin"], ["Río Grande"]])
```

Ventaja: no impone un orden. **Desventajas:** muchas categorías generan muchas columnas; las poco frecuentes tienen poca evidencia. Con `handle_unknown="ignore"`, una categoría nueva produce ceros en el bloque correspondiente: evita un error, pero no aprende qué significa esa categoría.

Algunos modelos lineales con intercepto necesitan una categoría de referencia para evitar redundancia exacta; `drop="first"` es una opción, no una regla universal. En esta clase conservamos todas las columnas. No hace falta estandarizar automáticamente estos indicadores.

Referencia: OneHotEncoder.

11.3 Alternativas de pandas

`Series.map({"no": 0, "sí": 1})` permite una codificación binaria explícita. **Ventaja:** se lee fácilmente. **Limitación:** una etiqueta no incluida produce un faltante; hay que comprobarlo.

`pd.get_dummies()` permite explorar one-hot sobre una tabla. **Ventaja:** es directo y devuelve un DataFrame. **Limitación:** ejecutarlo de forma independiente sobre entrenamiento y prueba puede crear columnas distintas. Es necesario alinear el esquema y definir qué sucede con categorías nuevas. Para un flujo reutilizable preferimos un `OneHotEncoder` ajustado en entrenamiento.

11.4 La salida de clasificación: `LabelEncoder()`

`LabelEncoder` se utiliza con `y`, cuando se necesitan identificadores numéricos de clases. No es la herramienta general para codificar columnas nominales de `X`.

```
from sklearn.preprocessing import LabelEncoder
etiquetas = LabelEncoder()
y_codificada = etiquetas.fit_transform(["renueva", "no renueva", "renueva"])
etiquetas.inverse_transform(y_codificada)
```

Ventajas: proporciona una correspondencia reversible. **Desventajas:** no expresa jerarquía; no admite sin más una clase nueva en prueba. Muchos clasificadores de scikit-learn aceptan directamente etiquetas de texto, de modo que no siempre se necesita. Se conserva la correspondencia para interpretar resultados.

Referencia: LabelEncoder.

12. Variables categóricas en regresión y clasificación

Regresión

El objetivo es una cantidad, como gasto futuro. Las entradas pueden mezclar ingreso, edad, ciudad y nivel. Imputamos y, si corresponde, escalamos las entradas numéricas; representamos las categóricas con ordinal, one-hot o una capacidad nativa del algoritmo. `y` conserva su significado cuantitativo y aquí se mantiene en pesos.

Convertir ciudad en indicadores no altera el tipo de problema. Una eventual transformación de `y` requeriría un tratamiento separado y una transformación inversa; queda fuera de esta introducción.

Clasificación

El objetivo es una clase: renueva / no renueva, o web / local / teléfono. Las entradas pueden ser las mismas que en regresión y su preparación sigue dependiendo de su significado y del algoritmo. La salida puede conservar etiquetas o usar códigos de clase. **No se estandarizan los códigos de clase como si fueran medidas.**

Una salida ordinal —por ejemplo, riesgo bajo, medio o alto— sigue siendo categórica. Codificarla 0/1/2 no hace que cualquier clasificador aproveche su orden ni justifica tratarla automáticamente como regresión.

Qué significa que un modelo trabaje con ciertos tipos de datos

No sería correcto decir que unos modelos son siempre «mejores con categorías» y otros «mejores con números». Hay que distinguir representación compatible, supuestos y rendimiento observado.

Orientación para próximas clases	Tipo de entrada y precaución
Métodos basados en distancias o cálculos numéricos	Requieren representaciones numéricas; codificar categorías y revisar escalas. Poder ejecutar no garantiza una distancia adecuada.
Árboles de decisión de scikit-learn	Pueden aprovechar información originalmente numérica y categórica codificada, pero `DecisionTreeClassifier` y `DecisionTreeRegressor` no aceptan directamente texto nominal en `X`. Suelen no necesitar escalado.
Implementaciones con soporte categórico nativo	Pueden tratar numéricas y categorías con procedimientos diferentes. Deben declararse tipos y respetarse los requisitos de la biblioteca.
Métodos diseñados para categorías, como `CategoricalNB`	Esperan entradas categóricas representadas con códigos enteros no negativos. Sus códigos se interpretan como categorías, no como cantidades. Las continuas necesitan otro tratamiento; discretizarlas implica perder detalle.

En un método explícitamente categórico, los códigos 0/1/2 pueden ser identificadores válidos de categorías nominales porque el algoritmo conoce ese significado. En un método numérico genérico pueden introducir distancias falsas. El problema no es el código aislado sino cómo lo interpreta el algoritmo.

Referencias: Árboles en scikit-learn, Naive Bayes categórico.

13. Entrenamiento, prueba y fuga de información

Reservamos una parte de los datos para comprobar posteriormente cómo funciona el modelo sobre observaciones no utilizadas para aprender. `train_test_split()` facilita esa separación. Para clasificación con suficientes casos por clase, `stratify=y` conserva aproximadamente las proporciones.

Ventaja: establece una evaluación separada. **Limitaciones:** una partición aleatoria no siempre es adecuada: series temporales requieren respetar el tiempo y observaciones de una misma persona pueden requerir separación por grupos. Aquí simulamos personas independientes.

`fit()` aprende parámetros; `transform()` aplica los parámetros aprendidos; `fit_transform()` hace ambas cosas sobre el conjunto indicado. Normalización por fila y binarización con umbral fijo no aprenden estadísticas globales, aunque conservan esta interfaz.

```
# Correcto: aprender en entrenamiento y reutilizar.
escalador.fit(X_train)
X_train_escalado = escalador.transform(X_train)
X_test_escalado = escalador.transform(X_test)
```

Calcular la mediana, la media o los extremos sobre toda la tabla antes de separar deja que la prueba participe de la preparación: es fuga de información. Ajustar otro escalador en prueba también está mal: cambia la referencia con la que se representan los datos. En validación cruzada futura, el ajuste debe repetirse dentro de cada partición de entrenamiento.

También hay fuga si incluimos en `X` el gasto futuro que queremos predecir, una etiqueta derivada de ese gasto o información que solo se conocerá después. La notebook excluye ambos objetivos de las entradas.

Referencia: Errores comunes y fuga de información.

14. Integrar con `Pipeline()` y `ColumnTransformer()`

`Pipeline` encadena pasos: por ejemplo, imputar y después estandarizar. `ColumnTransformer` aplica procesos distintos a grupos de columnas y reúne los resultados.

Ventajas: permiten repetir el mismo procedimiento y evitan olvidos al transformar datos nuevos. **Limitaciones:** requieren revisar la selección de columnas, el orden y los nombres de salida; no corrigen una decisión conceptual equivocada. Su uso adecuado reduce errores de fuga, pero no detecta por sí mismo una variable que contiene información futura.

Para un algoritmo sensible a escalas podemos preparar numéricas con imputación y estandarización, nominales con imputación y one-hot, ordinales con imputación y codificación explícita. Para un árbol podríamos conservar las numéricas sin escalar. La notebook muestra ambas variantes sin entrenar modelos.

15. Tabla de decisiones

Necesidad	Herramienta	Qué conserva / qué puede perder
Completar faltantes	`SimpleImputer`	Conserva filas; introduce valores estimados
Llevar columnas a referencia 0–1	`MinMaxScaler`	Conserva orden; los extremos comprimen el resto
Media 0 y desviación 1	`StandardScaler`	Conserva forma; sensible a extremos

Escalar con menor influencia de extremos	`RobustScaler`	Conserva extremos; no fija un rango
Comparar perfiles por fila	`Normalizer`, `normalize`	Conserva dirección; pierde magnitud
Detectar presencia o superar un umbral	`Binarizer`	Conserva la condición; pierde cantidad
Representar orden conocido	`OrdinalEncoder`	Conserva orden; puede imponer distancias
Representar categorías sin orden	`OneHotEncoder`, `get_dummies`	Evita jerarquía; aumenta columnas
Representar sí/no	`map`	Correspondencia explícita; revisar desconocidos
Codificar etiquetas de salida	`LabelEncoder`	Clases reversibles; no es un escalador
Aplicar todo de forma consistente	`Pipeline`, `ColumnTransformer`	Reproduce decisiones; requiere diseño correcto

4. Cierre:

Preparar datos implica tomar decisiones sobre su significado. La transformación correcta depende de las variables, la pregunta y el algoritmo. Al finalizar, debemos poder producir entradas compatibles y reproducibles sin usar información del futuro ni del conjunto de prueba para aprender su preparación.

En las próximas clases estudiaremos los modelos y evaluaremos cómo estas decisiones influyen en sus resultados.

5. Participación en Foro Obligatorio

¿Cómo prepararías estos datos?

Una empresa quiere utilizar datos de sus clientes para un futuro proyecto de aprendizaje automático. Antes de comenzar, encuentra esta tabla:

Cliente	Edad	Ingreso mensual	Ciudad	Satisfacción	Compras
A	25	800.000	Ushuaia	Alta	0
B	40	Sin dato	Tolhuin	Baja	3
C	35	1.200.000	Río Grande	Media	15
D	50	950.000	Ushuaia	Alta	1

Tu participación

Elegí **una** de estas situaciones:

- **Ingreso faltante:** ¿cómo lo completarías? ¿Qué inconveniente podría tener tu decisión?
- **Edad e ingreso:** si necesitamos comparar distancias entre clientes, ¿qué transformación aplicarías y por qué?
- **Ciudad:** ¿cómo la convertirías en números sin establecer un orden entre ciudades?
- **Satisfacción:** ¿cómo representarías Baja, Media y Alta? ¿Qué precaución deberíamos tener al interpretar esos números?
- **Compras:** si solo interesa saber si el cliente compró alguna vez, ¿qué transformación usarías? ¿Qué información perderíamos?

Escribí una respuesta que incluya:

1. La situación elegida y la función o técnica que aplicarías.
2. Una explicación con tus palabras de por qué resulta adecuada.
3. Una limitación o un caso en el que no convendría utilizarla.

Intercambio con un compañero

Respondé a una publicación de otro estudiante. Podés proponer una alternativa, señalar una precaución o plantear una pregunta que ayude a revisar su decisión. Explicá tu aporte: no alcanza con escribir «estoy de acuerdo».

No hace falta escribir código. Lo importante es justificar las decisiones. Puede haber más de una respuesta válida, según lo que necesitemos conservar de los datos.

Se valorará la claridad de la explicación, el uso correcto de los conceptos de la clase y el aporte al intercambio.

6. Material de consulta

Los enlaces específicos aparecen junto a cada técnica. Para ampliar:

- Guía de preprocesamiento.
- Imputación de faltantes.
- Composición de transformadores.
- Documentación de pandas.

Te esperamos en los próximos encuentros. Recordá que es importante participar de los foros, de los encuentros sincrónicos y en lo posible de las tutorías presenciales.

5. Bibliografía Obligatoria

- HAN, R. (2019): *Matemáticas del Aprendizaje Automático. Introducción a la analítica de datos e Inteligencia Artificial.*
- JONES, H. (2019): *Aprendizaje Automático. El Aprendizaje Automático para principiantes que desean comprender aplicaciones, Inteligencia Artificial, Minería de Datos, Big Data y más*. Editorial Edición Kindle.
- RUSSELL, R. (2018): *Machine Learning. Guia Paso a Paso Para Implementar Algoritmos De Machine Learning Con Python*
- VIDALES, A. (2019) *Machine Learning Con Matlab*. Editorial Edición Kindle.
- CANE, A. (2019): *Programación Con Python. Guía completa para principiantes.*
- HERNÁNDEZ, V. (2019): *Aprendizaje Automático. Implementación en Inteligencia Artificial.* Editorial Edición Kindle.

SUGERIDA

- "Introduction to TensorFlow." *TensorFlow*, https://www.tensorflow.org/learn. Accedido el 23 Junio del 2023.
- "IPython." *Jupyter and the future of IPython — IPython*, http://ipython.org/. Accedido el 23 Junio del 2023.
- "Matplotlib." *Matplotlib — Visualization with Python*, https://matplotlib.org/. Accedido el 23 Junio del 2023.
- "NumPy.org." *NumPy.org*, http://www.numpy.org/. Accedido el 23 Junio del 2023.
- "pandas." *pandas - Python Data Analysis Library*, https://pandas.pydata.org/. Accedido el 23 Junio del 2023.
- "Requests: HTTP para Humanos." *Requests: HTTP para Humanos — documentación de Requests - 1.1.0*, http://es.python-requests.org/es/latest/. Accedido el 23 Junio del 2023.
- "scikit-learn." *scikit-learn: machine learning in Python — scikit-learn 1.2.2 documentation*, https://scikit-learn.org/stable/. Accedido el 23 Junio del 2023.
- "seaborn." *seaborn: statistical data visualization — seaborn 0.12.2 documentation*, https://seaborn.pydata.org/. Accedido el 23 Junio del 2023.